Architectural patterns

The Model-View-Controller (MVC) pattern

The **Model-View-Controller (MVC)** framework is an architectural/design pattern that separates an application into three main logical components **Model**, **View**, and **Controller**. Each architectural component is built to handle specific development aspects of an application. It isolates the business logic and presentation layer from each other.

The Model-View-Controller (MVC) pattern is an architectural pattern that separates the concerns of data management, user interface, and user input control. It consists of three main components:

Controller:

The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary. The controller doesn't have to worry about handling data logic, it just tells the model what to do. It processes all the business logic and incoming requests, manipulates data using the **Model** component, and interact with the **View** to render the final output.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

- Model: Represents the data and business logic of the application.
- View: Represents the user interface and displays the data from the model.
- Controller: Handles user input and updates the model and view accordingly.

View:

The **View** component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren't taken directly but through the controller. It only interacts with the controller.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.

Model:

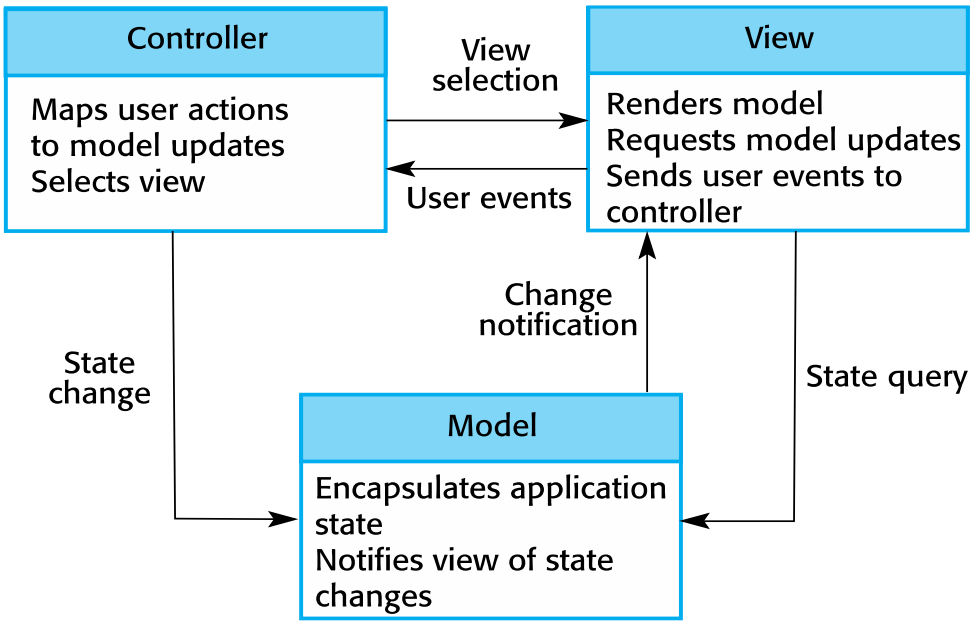
The **Model** component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. It can add or retrieve data from the database. It responds to the controller's request because the controller can't interact with the database by itself. The model interacts with the database and gives the required data back to the controller.

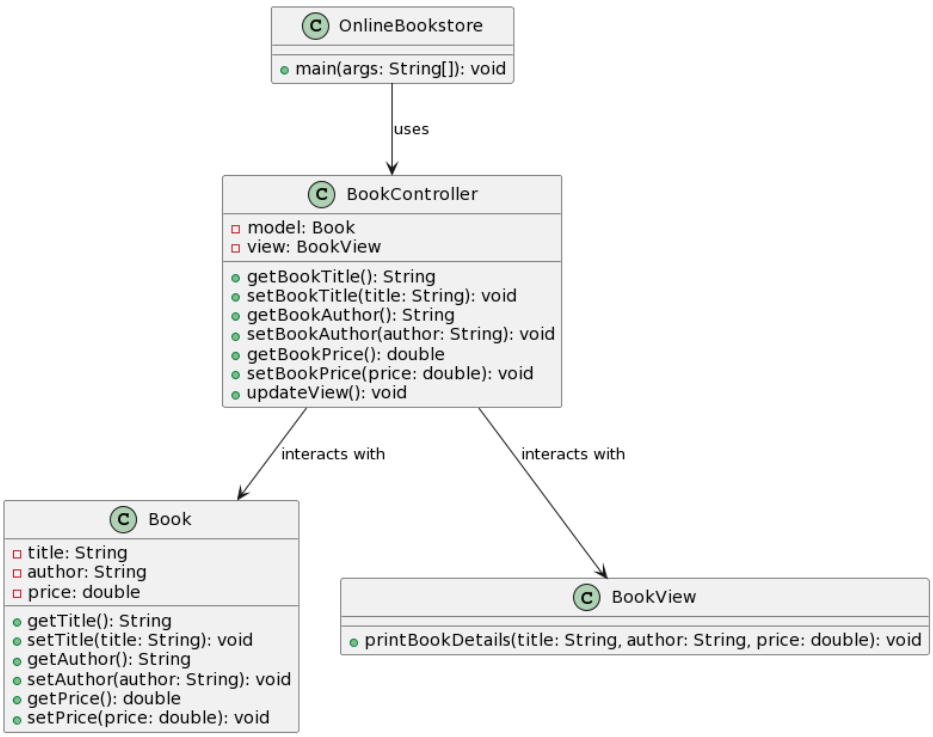
Responsibilities:

- Managing data: CRUD (Create, Read, Update, Delete) operations.

- Enforcing business rules.
- Notifying the View and Controller of state changes.

Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level.





Layered Architecture (N-Tier Architecture)

Layered Architecture (N-Tier Architecture) is a software design pattern that structures an application into multiple distinct layers, each responsible for specific tasks or concerns. This approach helps in separating different aspects of the application into modular, manageable, and reusable components. Each layer interacts with the one directly above or below it, but layers typically don't interact with each other directly, promoting a clear separation of concerns.

Repository Design Pattern: Simplifying Data Access

The Repository Design Pattern is a software design pattern that acts as an intermediary layer between an application's business logic and data storage.

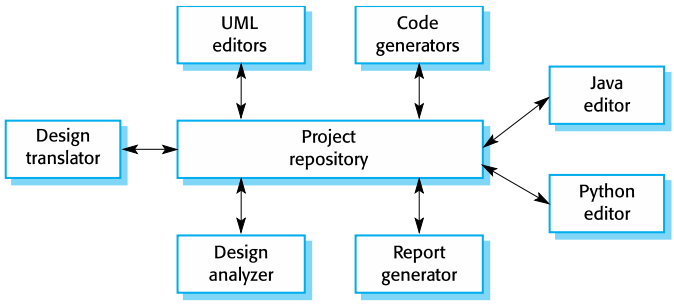
- Its primary purpose is to provide a structured and standardized way to access, manage, and manipulate data while abstracting the underlying details of data storage technologies.
- This pattern promotes a clear separation of concerns, making software more maintainable, testable, and adaptable to changes in data sources.

Name	Repository
Description	All data in a system is managed in a central repository that is accessible to all system components. Components do not interact directly, only through the repository.
Example	Figure 6.9 is an example of an IDE where the components use a repository of system design information. Each software tool generates information which is then available for use by other tools.
When used	You should use this pattern when you have a system in which large volumes of information are generated that has to be stored for a long time. You may also use it in data-driven systems where the inclusion of data in the repository triggers an action or tool.
Advantages	Components can be independent—they do not need to know of the existence of other components. Changes made by one component can be propagated to all components. All data can

A

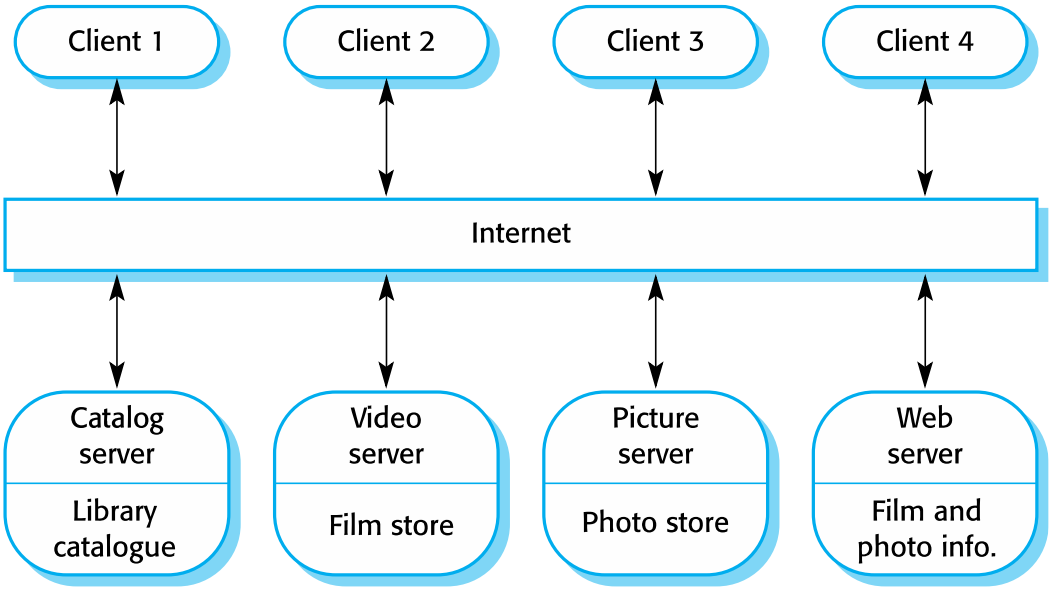
	be managed consistently (e.g., backups done at the same time) as it is all in one place.
Disadvantages	The repository is a single point of failure so problems in the repository affect the whole system. May be inefficiencies in organizing all communication through the repository. Distributing the repository across several computers may be difficult.

repository architecture for an IDE



Client-server architecture

Client-server architecture is a fundamental concept in system design where a network involves multiple clients and a server. Clients are devices or programs that request services or resources, while the server is a powerful machine providing these resources or services. This architecture allows efficient data management and resource sharing, making it popular in web applications, databases, and other network-based systems. By separating roles and distributing tasks, client-server architecture enhances performance, scalability, and security.



The pipe and filter pattern

In system design, the Pipe and Filter architecture structures processing tasks into a sequence of stages known as "pipes," where each stage filters and transforms data incrementally. This modular framework enables filters to operate independently, improving scalability and reusability. Each filter is tasked with specific operations, such as validating or formatting data, and passes its output to the next stage via interconnected pipes. This architecture fosters flexibility and ease of maintenance by isolating concerns and allowing components to be reused across various systems.

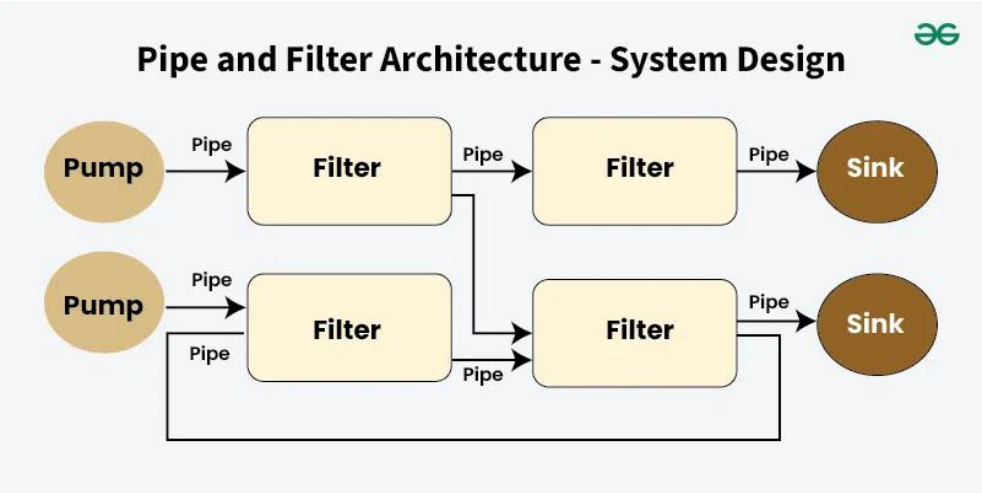
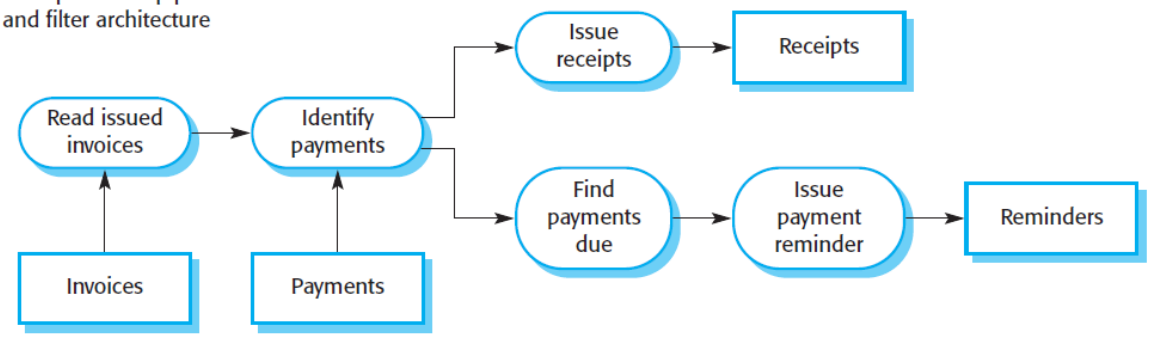


Figure 6.15 An example of the pipe and filter architecture



Pipe and filter systems are best suited to batch processing systems and embedded systems where there is limited user interaction. Interactive systems are difficult to write using the pipe and filter model because of the need for a stream of data to be processed.